

Tecnologías SI.

Capa de persistencia (II)

Java Persistence API (JPA) – Ampliación

MEI-SI, 2018

Octubre 2018

Contenido

Mapeo de la herencia en JPA

Validaciones Bean Validation

Consultas JPQL

Criteria API

TAREAS PARA PRÓXIMA SEMANA

Mapeo de herencia en JPA I

Posibilidad de **mapeo de herencia** entre entidades.

- ▶ *Superclase* puede ser o no una entidad JPA
 - ▶ Superclase (abstracta o concreta) es una entidad JPA mapeada
 - ▶ Superclase no es entidad JPA, pero porta información de mapeo
 - ▶ superclase marcada con `@MappedSuperclass`
 - ▶ subclasses son entidades JPA y reutilizan el mapeo indicado en la superclase
 - ▶ Superclase no entidad JPA
 - ▶ superclase "normal" sin información de mapeo
 - ▶ JPA no maneja directamente esas información
- ▶ Entidades JPA pueden ser extendidas con subclasses no entidad

Mapecto de herencia en JPA II

Control de la herencia en JPA

- ▶ Anotación `@Inheritance(strategy='...')` en la entidad *superclase* controla el modo de mapeo de la herencia
 - ▶ `InheritanceType.SINGLE_TABLE`: una única tabla común para *superclase* y *subclase* (con "nulos")
 - ▶ `InheritanceType.TABLE_PER_CLASS`: una tabla por *subclase* concreta (incluye los atributos heredados y los propios)
 - ▶ `InheritanceType.JOINED`: una tabla por clase, tanto *superclase* (atributos heredados) como *subclase* (atributos propios)
- ▶ Control del mapeo de herencia con `@DiscriminatorColumn` (en *superclase*) y `@DiscriminatorValue` (en *subclases*)
- ▶ Anotación `@MappedSuperclass` usada en *superclases* que no son entidades JPA (no mapeadas)
 - ▶ Usa una estrategia equivalente a `TABLE_PER_CLASS`
 - ▶ Posibilidad de incluir anotaciones de mapeo JPA en sus atributos ("heredadas" por sus *subclases* entidad)

Mapecto de herencia en JPA III

Ejemplos

```
@Entity
@Inheritance(strategy=[TipoEstrategia])
public abstract class Persona {
    @Id long id;
    String nombre;
    ...
}
```

```
@Entity
public class Profesor extends Persona {
    @Enumerated TipoProfesor tipo;
    Double sueldo;
    @ManyToOne List<Materia> materias;
    ...
}
```

```
@Entity
public class Alumno extends Persona {
    @ManyToOne List<Materia> materias;
    ...
}
```

```
@MappedSuperclass
public abstract class Persona {
    @Id long id;
    String nombre;
    ...
}
```

```
@Entity
public class Profesor extends Persona {
    @Enumerated TipoProfesor tipo;
    Double sueldo;
    @ManyToOne List<Materia> materias;
    ...
}
```

```
@Entity
public class Alumno extends Persona {
    @ManyToOne List<Materia> materias;
    ...
}
```

[TipoEstrategia]

SINGLE_TABLE

TABLE_PER_CLASS

JOINED

1 tabla común con todos los atributos

2 tablas (*Profesor*, *Alumno*)

3 tablas (*Persona*, *Profesor*, *Alumno*)

2 tablas (*Profesor*, *Alumno*)

Contenido

Mapeo de la herencia en JPA

Validaciones Bean Validation

Consultas JPQL

Criteria API

TAREAS PARA PRÓXIMA SEMANA

Validaciones Bean Validation

Bean Validation ofrece un framework para definición de metadatos sobre objetos Java y para la validación de los mismos

- ▶ Proporciona anotaciones para vincular restricciones sobre los atributos de objetos Java (incluyendo Entidades JPA)
- ▶ Automatiza la validación de estas restricciones
 - ▶ Disponible en las diversas capas de la aplicación
 - ▶ En capa de persistencia para validar la corrección de las entidades JPA
 - ▶ En capa de presentación para validar las entradas de usuario
- ▶ Permite la definición de anotaciones de validación personalizadas
- ▶ Definido en JSR 303, ver <https://beanvalidation.org/>
- ▶ Implementación de referencia: Hibernate Validator

URL: <http://hibernate.org/validator/>

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-validator</artifactId>
  <version>6.0.13.Final</version>
</dependency>
```

Aclaración: Las anotaciones Bean Validation son complementarias a las de los mapeos JPA (no las reemplazan).

```
@Entity
public class Entidad {
    @Column(length=10)
    @Size(max = 10)
    private String campo;
    ...
}
```

- ▶ @Column especifica detalles del mapeo JPA (afecta a la generación de código DDL de la correspondiente tabla)
- ▶ @Size establece una restricción respecto al tamaño del atributo

Anotaciones Bean Validation

Vinculadas a la definición de atributos, métodos `set()` o parámetros de métodos

<code>@Min(v)</code>	Valor debe ser menor o igual que el indicado
<code>@Max(v)</code>	Valor debe ser menor o igual que el indicado
<code>@DecimalMax(v)</code>	Valor decimal debe ser menor o igual que el indicado
<code>@DecimalMin(v)</code>	Valor decimal debe ser mayor o igual que el indicado
<code>@Digit(integer=i, fraction=f)</code>	Longitud valor numérico debe coincidir con especificaciones
<code>@Future</code>	El valor de una fecha debe situarse en el futuro
<code>@Past</code>	El valor de una fecha debe situarse en el pasado
<code>@NotNull</code>	El valor no puede ser null
<code>@Null</code>	El valor debe ser null
<code>@NotBlank</code>	El valor de la cadena debe contener al menos un caracter no espacio
<code>@Pattern(regexp="p")</code>	El valor de la cadena debe coincidir con el patrón indicado
<code>@Size(min=i,max=x)</code>	El tamaño (de una cadena, de una colección, de un array) debe cumplir la restricción indicada
<code>@NotEmpty</code>	La (cadena, colección, array) no puede estar vacía
<code>@AssertTrue</code>	El valor debe ser <i>true</i>
<code>@AssertFalse</code>	El valor debe ser <i>false</i>

Otras: `@PastOrPresent`, `@FutureOrPresent`, `@Positive`, `@Negative`, `@Email`, ...

Más detalles (Java EE 8 tutorial):

<https://javaee.github.io/tutorial/bean-validation002.html>

Validaciones definidas por usuario

Posibilidad de definir validaciones propias

- ▶ Aplicables sobre atributos (anotan definición de atributos) o sobre objetos completos (anotan las clases)
- ▶ Supone definir:
 1. Una **anotación propia**
 - ▶ que herede de `@Document`
 - ▶ que declare `@Constraint(validatedBy=ClaseValidadora.class)`
 2. Una **clase validadora**
 - ▶ que implemente la interface `ConstraintValidator<>`
 - ▶ debe definir los métodos `initialize()` y `isValid()`

Ejemplo Validación propia @Nif

```
@Target({ METHOD, FIELD, ANNOTATION_TYPE })
@Retention(RUNTIME)
@Constraint(validatedBy = NifValidator.class)
@Documented
public @interface Nif {
    String message() default "{constraint nif}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```
@Entity
public class Usuario {
    @Id
    @Nif
    private String nif;
    ...
}
```

```
public class NifValidator implements
    ConstraintValidator<Nif, String> {
    private String letters = "TRWAGMYFPDXBNJZSQVHLCKE";
    private Pattern mask = Pattern.compile("[0-9]{8,8}[A-Z]");

    @Override
    public void initialize(Nif constraintAnnotation) {}

    @Override
    public boolean isValid(String value,
        ConstraintValidatorContext context) {
        Matcher matcher = mask.matcher(value);
        if (!matcher.matches()){
            return false;
        }

        String dni = value.substring(0,8);
        String control = value.substring(8,9);
        int position = Integer.parseInt(dni)%23;

        String controlCalculated =
            letters.substring(position,position+1);

        if (!control.equalsIgnoreCase(controlCalculated)){
            return false;
        }
        return true;
    }
}
```

Contenido

Mapeo de la herencia en JPA

Validaciones Bean Validation

Consultas JPQL

Criteria API

TAREAS PARA PRÓXIMA SEMANA

Consultas JPQL I

JPQL: *Java Persistence Query Language*

- ▶ Ofrece sintaxis similar a la de SQL para realizar consultas sobre entidades JPA
- ▶ Clausula FROM maneja nombres de Entidades (no tablas)
 - ▶ Hace uso de las relaciones mapeadas 1:N o N:M para definir los JOINS entre tablas relacionadas
 - ▶ Sintaxis: IN ó JOIN
 - ▶ No necesario con relaciones 1:1 o N:1
- ▶ Nombres de campos usan notación Java: *Entidad.atributo*
 - ▶ Los elementos manejados en WHERE o devueltos por SELECT son siempre objetos
- ▶ Se permiten consultas de tipo SELECT, UPDATE y DELETE

Consultas JPQL II

Consultas (objetos de tipo *Query* ó *TypedQuery* desde JPA 2.1) son creadas por el EM

`Query createQuery(String query)` : crea una consulta JPQL

`Query createNamedQuery(String namedQuery)` : crea una consulta nombrada, vinculada a una Entidad y definida con la anotación

`@NamedQuery(name="...", queryString="...")`

`Query createNativeQuery(String query)` : crea una consulta SQL

Nota: Para la creación de *TypedQuery* la sintaxis es la misma, añadiendo el tipo de la clase que devolverá la consulta (evita un *cast* explícito).

```
TypedQuery<Articulos> query = em.createQuery("SELECT a FROM Articulo a", Articulo.class);
```

Consultas JPQL III

Ejecución de consultas

`Object getSingleResult()` : devuelve un objeto con el resultado único de la consulta

- ▶ Devuelve un array (`Object[]`) cuando SELECT especifica varios valores

`List getResultList()` : devuelve una colección *List* con los resultados de la consulta

- ▶ Se puede especificar el rango de valores a devolver (útil para paginar resultados)
- ▶ Query `setFirstResult(int)`: inicio del rango
- ▶ Query `setMaxResults(int)`: tamaño del rango

`int executeUpdate()` : ejecución de Queries de tipo UPDATE o DELETE

Nota: Con TypedQuery los valores devueltos (simple o lista) estan tipados según se indicara en la creación de la TypedQuery.

Consultas JPQL IV

Consultas parametrizadas

- ▶ Notación: $\left\{ \begin{array}{l} \text{:nombre (parámetros con nombre)} \\ \text{?3 (parámetros posicionales)} \end{array} \right.$
- ▶ Métodos de la clase *Query*
 - `Query setParameter(String s, Object o)` : asocia valor a un parámetro nombrado
 - `Query setParameter(int pos, Object o)` : asocia valor a un parámetro posicional

Ejemplos:

- ▶ Lista de artículos cuya descripción empiece por CPU, "paginados" de 10 en 10

```
TypeQuery<Articulo> q1 = em.createQuery("SELECT a FROM Articulo a "+
                                         "WHERE a.descripcion LIKE \"CPU%\" ", Articulo.class);

List<Articulo> lista;
int i = 0;
do {
    q1.setFirstResult(i);
    q1.setMaxResult(10);
    lista = q1.getResultList();
    i = i + 10;
    ...
} while (lista !=null);
```

Consultas JPQL V

► Lista de pares (Pedido, importe total pedido) entre 2 fechas

```
Query q2 = em.createQuery("SELECT p, SUM(l.cantidad * l.articulo.precio)"+
    "FROM Pedido p JOIN p.lineas"+
    "WHERE p.fecha BETWEEN :fechaIni AND :fechaFin"+
    "GROUP BY p.numPedido);
q2.setParameter("fechaIni", new Date(...));
q2.setParameter("fechaFin", new Date(...));
Iterator pares = q2.getResultList().iterator();
for(Object[] par: pares) {
    pedido = (Pedido) par[0];
    total = (Float) par[1];
    ...
}
```


Contenido

Mapeo de la herencia en JPA

Validaciones Bean Validation

Consultas JPQL

Criteria API

TAREAS PARA PRÓXIMA SEMANA

Criteria API I

pendiente